Desenvolvimento de Apps para Mobile

Fabrício Tonetto Londero

Aula 14 – Flutter



Alta Reutilização

Compartilhe até 90% do código entre plataformas.

Eficiência

Reduza custos e tempo de desenvolvimento drasticamente.

Experiência Unificada

iOS e Android simultâneos sem duplicação de código.



Flutter

Experiência Unificada

iOS e Android simultâneos sem duplicação de código.

Multiplataforma

Desenvolve para iOS, Android, web e desktop com uma só base.



Alta Performance

Gráficos acelerados por hardware em todas plataformas.

Deploy Rápido

Publicação eficiente em Android, iOS, Windows, Mac e Linux.



Controle Total

Engine próprio para renderização pixel a pixel.

Uma Base só

Aplicações para mobile, web e desktop com único código.

UI Adaptativa

Interfaces que se ajustam automaticamente a qualquer tela.

Flutter



Integrações e Ecossistema

Firebase e APIs

Integração facilitada com backend e serviços externos.

Comunidade Ativa

Suporte constante e documentação abrangente.

Ferramentas Visuais

Crescimento de recursos como FlutterFlow para acelerar desenvolvimento.

Ecossistema em Evolução

Ferramentas e recursos ainda estão mudando rapidamente.

Ajustes de Performance

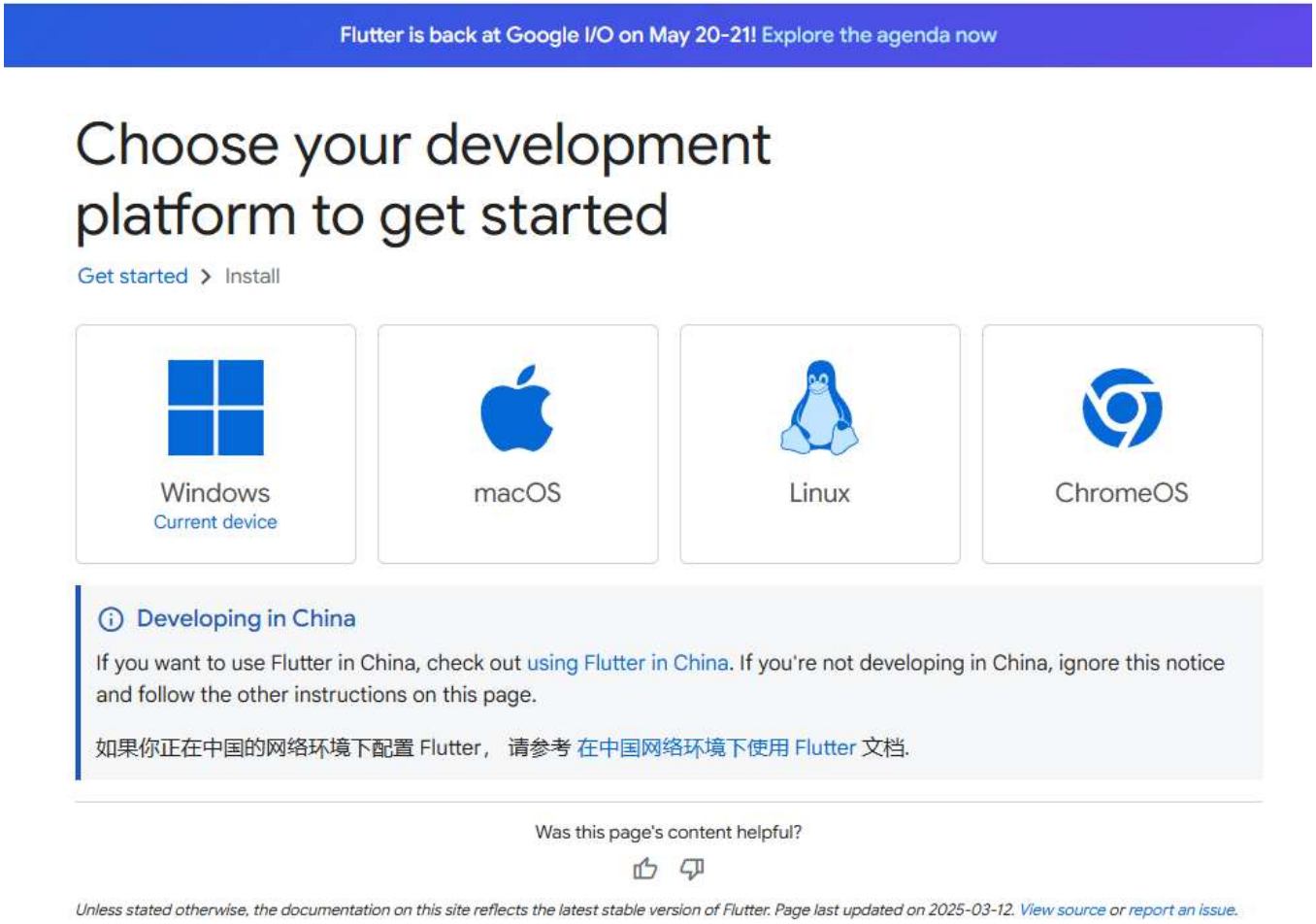
Apps complexos podem precisar otimizações pontuais.

Plugins Nativos

Alguns exigem código específico de cada plataforma.

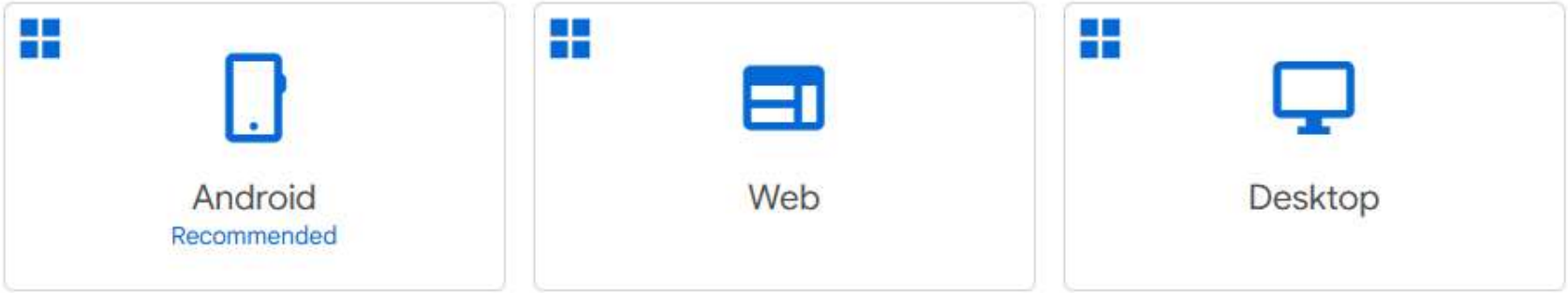
Desafios e Considerações

https://docs.flutter.dev/get-started/install



Choose your first type of app

Get started > Install > Windows



Your choice informs which parts of Flutter tooling you configure to run your first Flutter app. You can set up additional platforms later. *If you don't have a preference, choose **Android**.*

Developing in China

If you want to use Flutter in China, check out using Flutter in China. If you're not developing in China, ignore this notice and follow the other instructions on this page.

如果你正在中国的网络环境下配置 Flutter， 请参考 在中国网络环境下使用 Flutter 文档.

Was this page's content helpful?

Start building Flutter Android apps on Windows

Software requirements

To write and compile Flutter code for Android, you must have the following version of Windows and the listed software packages.

> **Tip**
>
> You do not need to install Dart separately as the Flutter SDK includes the full Dart SDK.

Operating system

Flutter supports 64-bit version of Microsoft Windows 10 or later. These versions of Windows should include the required Windows PowerShell 5 or later.

Development tools

Download and install the Windows version of the following packages:

- Git for Windows 2.27 or later to manage source code.
- Android Studio 2024.1.1 (Koala) or later to debug and compile Java or Kotlin code for Android. Flutter requires the full version of Android Studio.

The developers of the preceding software provide support for those products. To troubleshoot installation issues, consult that product's documentation.

Install the Flutter SDK

To install the Flutter SDK, you can use the VS Code Flutter extension or download and install the Flutter bundle yourself.

Use VS Code to install | Download and install

Use VS Code to install Flutter

To install Flutter using these instructions, verify that you have installed Visual Studio Code 1.86 or later and the Flutter extension for VS Code.

Prompt VS Code to install Flutter

1. Launch VS Code.
2. To open the **Command Palette**, press `Control` + `Shift` + `P`.
3. In the **Command Palette**, type `flutter`.
4. Select **Flutter: New Project**.
5. VS Code prompts you to locate the Flutter SDK on your computer.

Install the Flutter SDK

To install the Flutter SDK, you can use the VS Code Flutter extension or download and install the Flutter bundle yourself.

Use VS Code to install | Download and install

Download then install Flutter

To install Flutter, download the Flutter SDK bundle from its archive, move the bundle to where you want it stored, then extract the SDK.

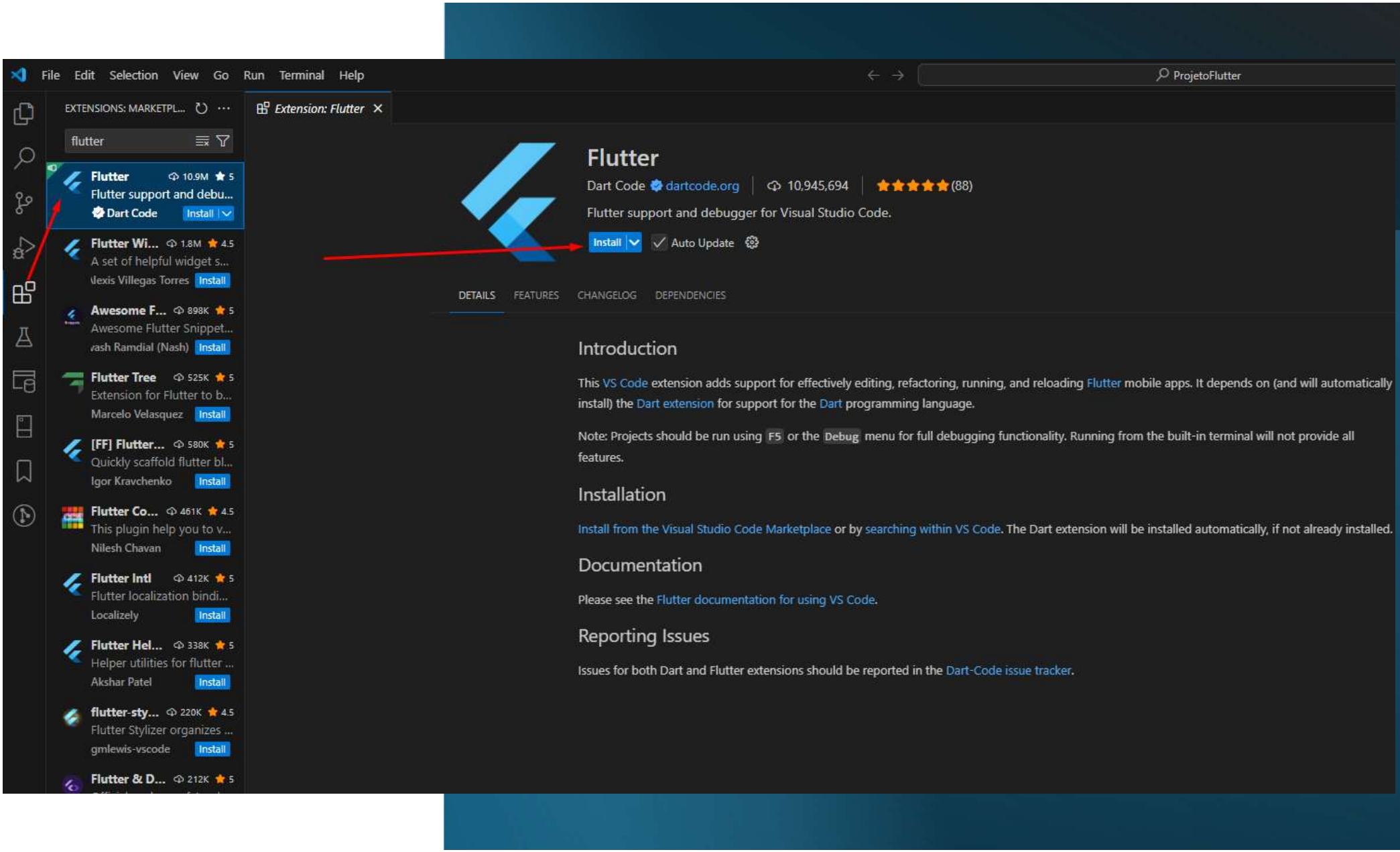
1. Download the following installation bundle to get the latest stable release of the Flutter SDK.

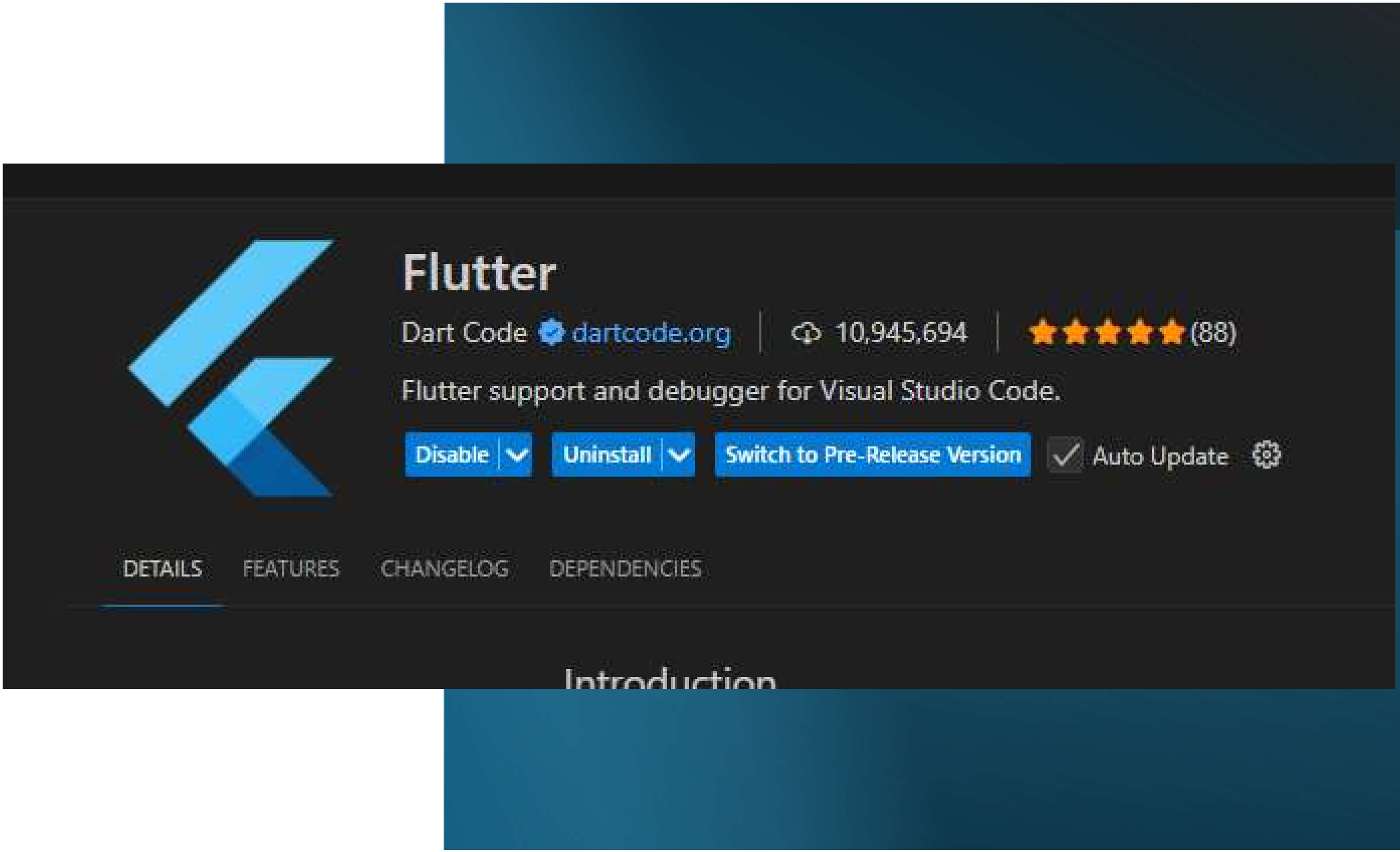
   flutter_windows_3.29.3-stable.zip

   For other release channels, and older builds, check out the SDK archive.

   The Flutter SDK should download to the Windows default download directory: `%USERPROFILE%\Downloads`.

   If you changed the location of the Downloads directory, replace this path with that path. To find your Downloads directory location, check out this Microsoft Community post.

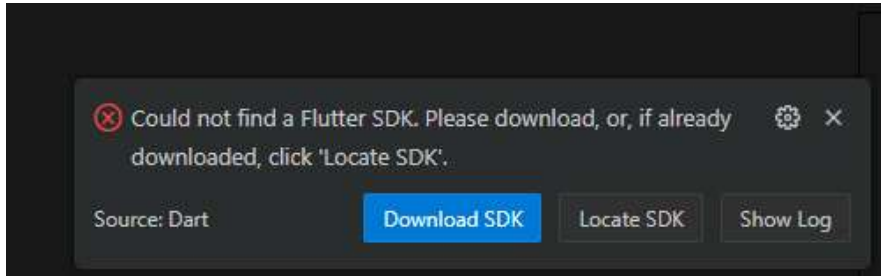

File Edit Selection View Go Run Terminal Help
ProjetoFlutter
EXTENSIONS: MARKETPL...
flutter
Extension: Flutter
Flutter
Flutter support and debu...
Dart Code
Install
Flutter Wi... 1.8M 4.5
A set of helpful widget s...
Alexis Villegas Torres
Awesome F... 898K 5
Awesome Flutter Snippet...
Nash Ramdial (Nash)
Flutter Tree 525K 5
Extension for Flutter to b...
Marcelo Velasquez
[FF] Flutter... 580K 5
Quickly scaffold flutter bl...
Igor Kravchenko
Flutter Co... 461K 4.5
This plugin help you to v...
Nilesh Chavan
Flutter Intl 412K 5
Flutter localization bindi...
Localizely
Flutter Hel... 338K 5
Helper utilities for flutter ...
Akshar Patel
flutter-sty... 220K 4.5
Flutter Stylizer organizes ...
gmlewis-vscode
Flutter & D... 212K 5
Flutter
Dart Code dartcode.org | 10,945,694 | (88)
Flutter support and debugger for Visual Studio Code.
Install
Auto Update
DETAILS FEATURES CHANGELOG DEPENDENCIES
Introduction
This VS Code extension adds support for effectively editing, refactoring, running, and reloading Flutter mobile apps. It depends on (and will automatically install) the Dart extension for support for the Dart programming language.
Note: Projects should be run using F5 or the Debug menu for full debugging functionality. Running from the built-in terminal will not provide all features.
Installation
Install from the Visual Studio Code Marketplace or by searching within VS Code. The Dart extension will be installed automatically, if not already installed.
Documentation
Please see the Flutter documentation for using VS Code.
Reporting Issues
Issues for both Dart and Flutter extensions should be reported in the Dart-Code issue tracker.

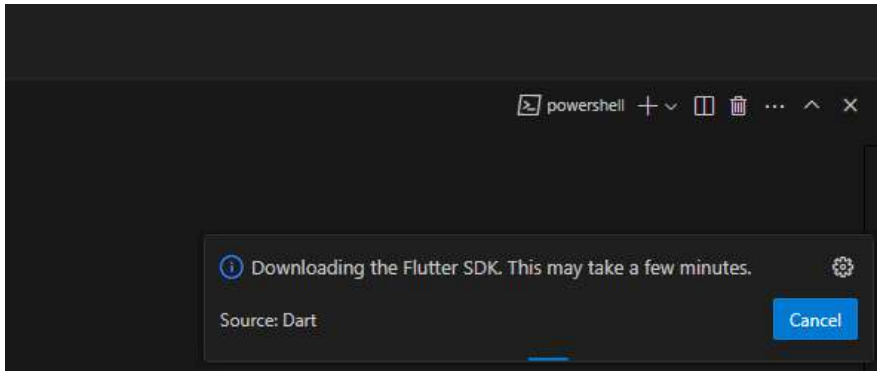

Flutter
Dart Code
dartcode.org
10,945,694
(88)
Flutter support and debugger for Visual Studio Code.
Disable
Uninstall
Switch to Pre-Release Version
Auto Update
DETAILS
FEATURES
CHANGELOG
DEPENDENCIES
Introduction

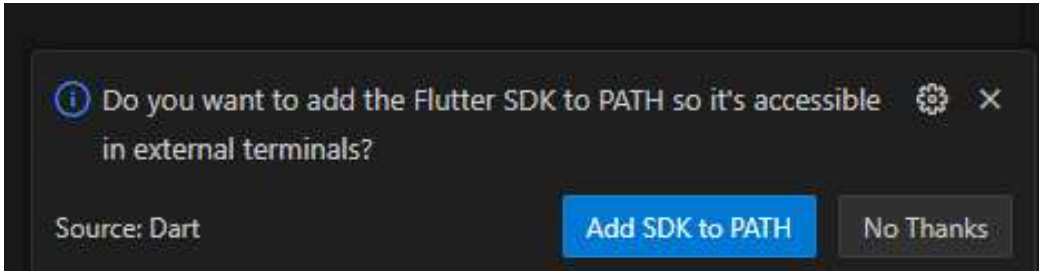
Prompt VS Code to install Flutter

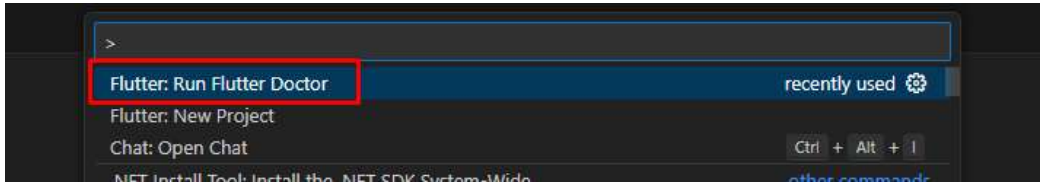
1. Launch VS Code.
2. To open the **Command Palette**, press `Control` + `Shift` + `P`.
3. In the **Command Palette**, type `flutter`.
4. Select **Flutter: New Project**.
5. VS Code prompts you to locate the Flutter SDK on your computer.
    1. If you have the Flutter SDK installed, click **Locate SDK**.
    2. If you do not have the Flutter SDK installed, click **Download SDK**.

       This option sends you the Flutter install page if you have not installed Git for Windows as directed in the development tools prerequisites.
6. When prompted **Which Flutter template?**, ignore it. Press `Esc`. You can create a test project after checking your development setup.

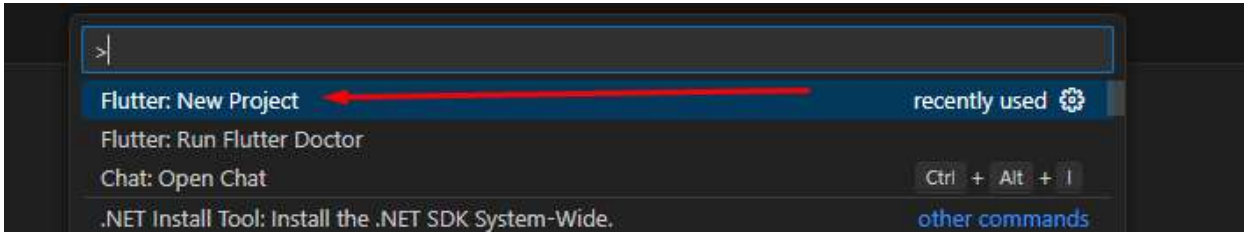

Could not find a Flutter SDK. Please download, or, if already downloaded, click 'Locate SDK'.
Source: Dart
Download SDK
Locate SDK
Show Log

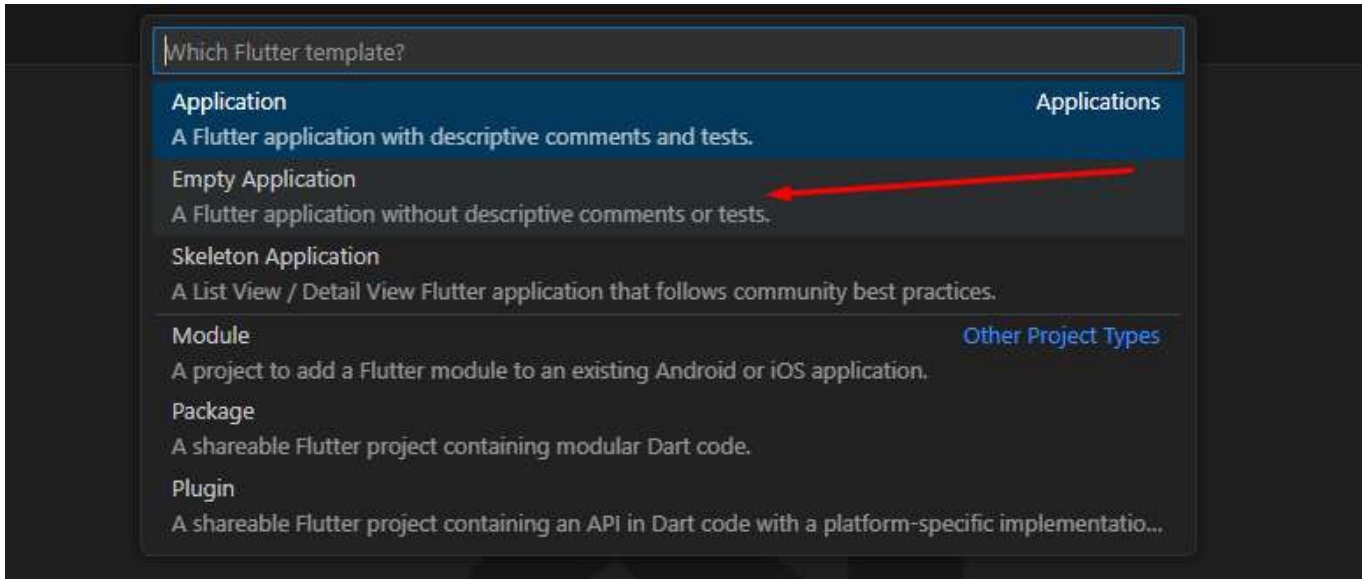

powershell
Downloading the Flutter SDK. This may take a few minutes.
Source: Dart
Cancel

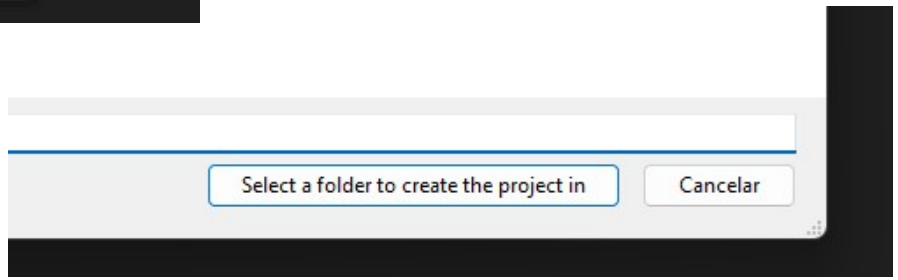

Do you want to add the Flutter SDK to PATH so it's accessible in external terminals?
Source: Dart
Add SDK to PATH
No Thanks

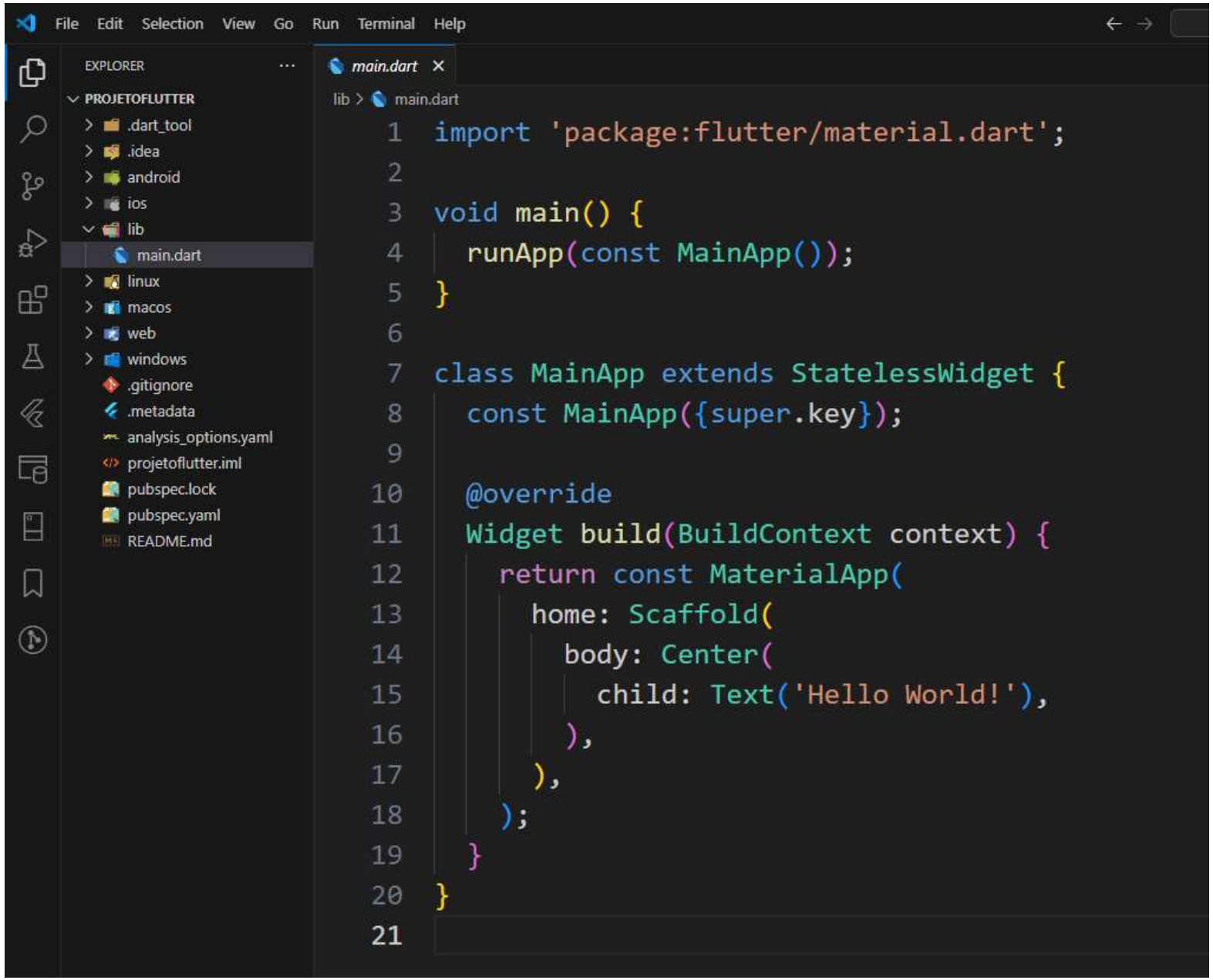

>
Flutter: Run Flutter Doctor
recently used
Flutter: New Project
Chat: Open Chat
Ctrl + Alt + I

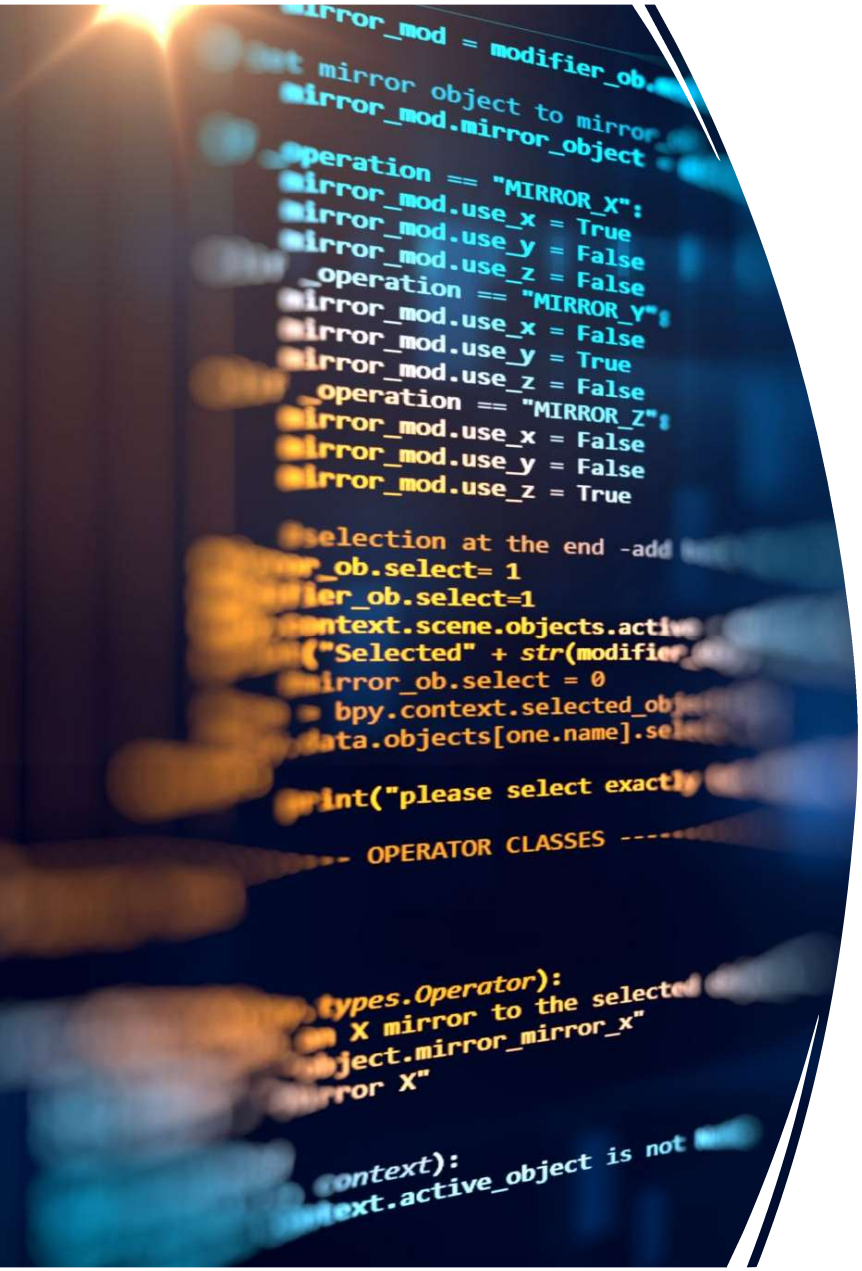
Criando o Projeto








File Edit Selection View Go Run Terminal Help
EXPLORER
PROJETOFLUTTER
.dart_tool
.idea
android
ios
lib
main.dart
linux
macos
web
windows
.gitignore
.metadata
analysis_options.yaml
projetoflutter.iml
pubspec.lock
pubspec.yaml
README.md
main.dart
lib > main.dart
1 import 'package:flutter/material.dart';
2
3 void main() {
4 runApp(const MainApp());
5 }
6
7 class MainApp extends StatelessWidget {
8 const MainApp({super.key});
9
10 @override
11 Widget build(BuildContext context) {
12 return const MaterialApp(
13 home: Scaffold(
14 body: Center(
15 child: Text('Hello World!'),
16),
17),
18);
19 }
20 }
21



Projeto Flutter

- lib/: Onde fica o código principal do app Flutter
  - main.dart: Arquivo inicial do app
- android/, ios/, linux/, macos/, windows/
  - Pastas que contêm o código específico para cada plataforma.
  - Não mexemos nelas no começo.
- web/
  - Código para rodar o app em navegador.
- pubspec.yaml
  - Onde declaramos dependências (bibliotecas externas), imagens, fontes etc.

Sobre o Código Dart

- import 'package:flutter/material.dart’;
  - Importa os componentes visuais (botões, textos, layouts) do **Material Design**, padrão do Flutter.

```
void main() {
  runApp(const MainApp());
}
```

  - É a função principal
    - Ponto de entrada do app.
  - runApp(...): Diz ao Flutter para mostrar a interface criada.
  - MainApp() é o widget (componente) principal que será exibido na tela.

Classe MainApp

class MainApp extends StatelessWidget {

const MainApp({super.key});

MainApp é um widget sem estado (StatelessWidget) — ou seja, a interface não muda com o tempo.

O *super.key* ajuda a Flutter identificar e otimizar atualizações no layout.

Método build

```
@override
Widget build(BuildContext context) {
  return const MaterialApp(
    home: Scaffold(
      body: Center(
        child: Text('Hello World!'),
      ),
    ),
  );
}
```

Esse trecho monta a interface que será exibida:

MaterialApp: Estrutura básica do app com o tema do Material Design.

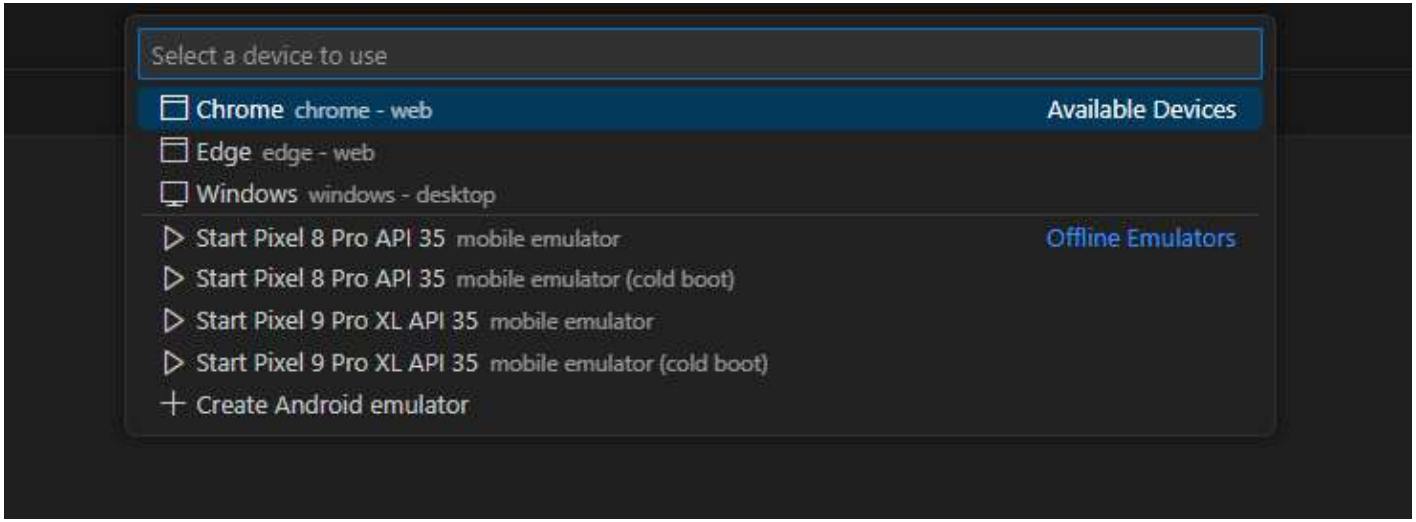
home: Tela principal do app.

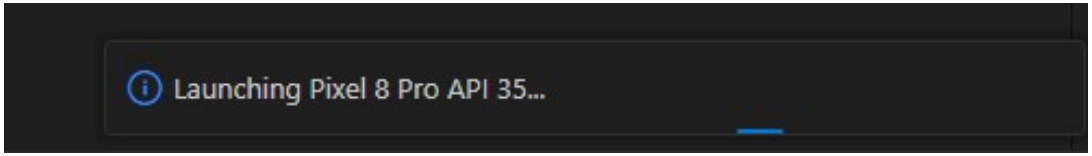
Scaffold: Proporciona uma estrutura de layout com barra superior, corpo, botão flutuante etc.

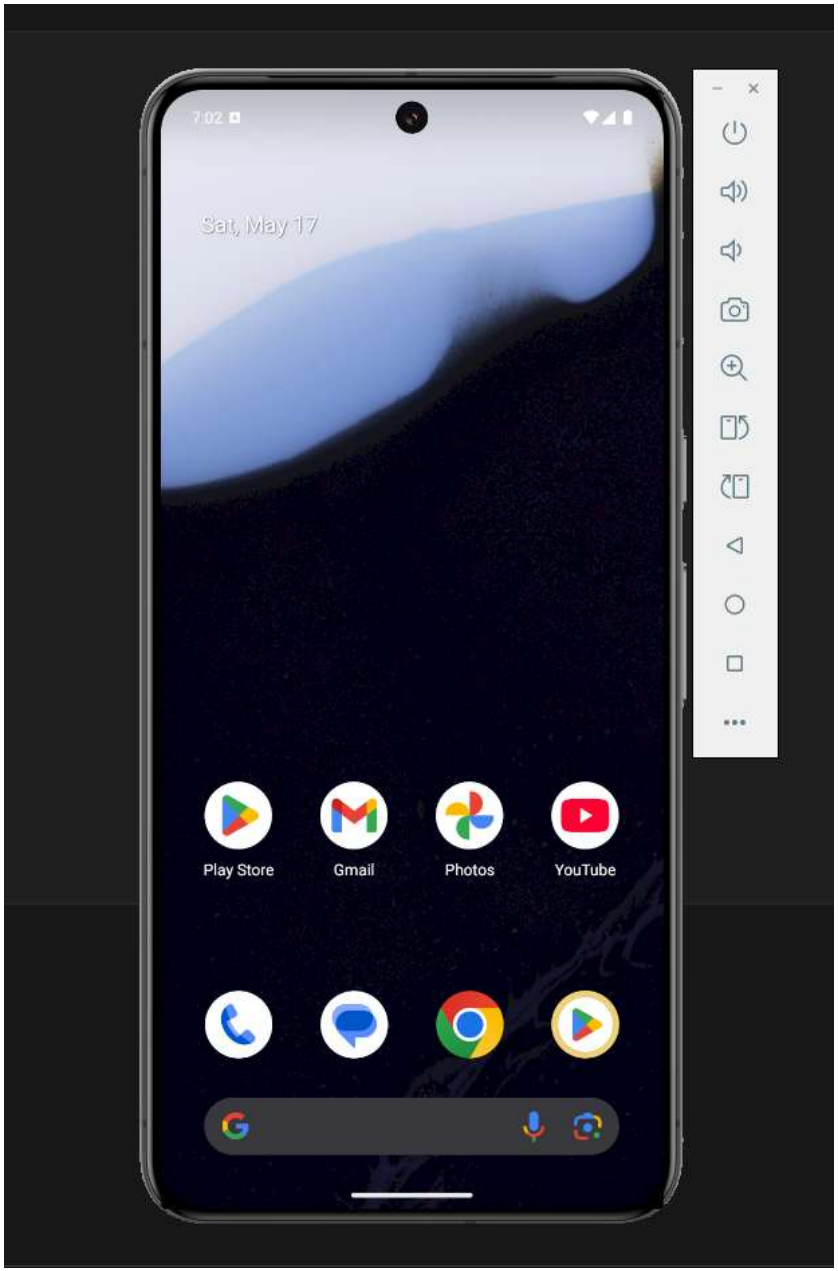
body: Center(...): Centraliza o conteúdo no meio da tela.

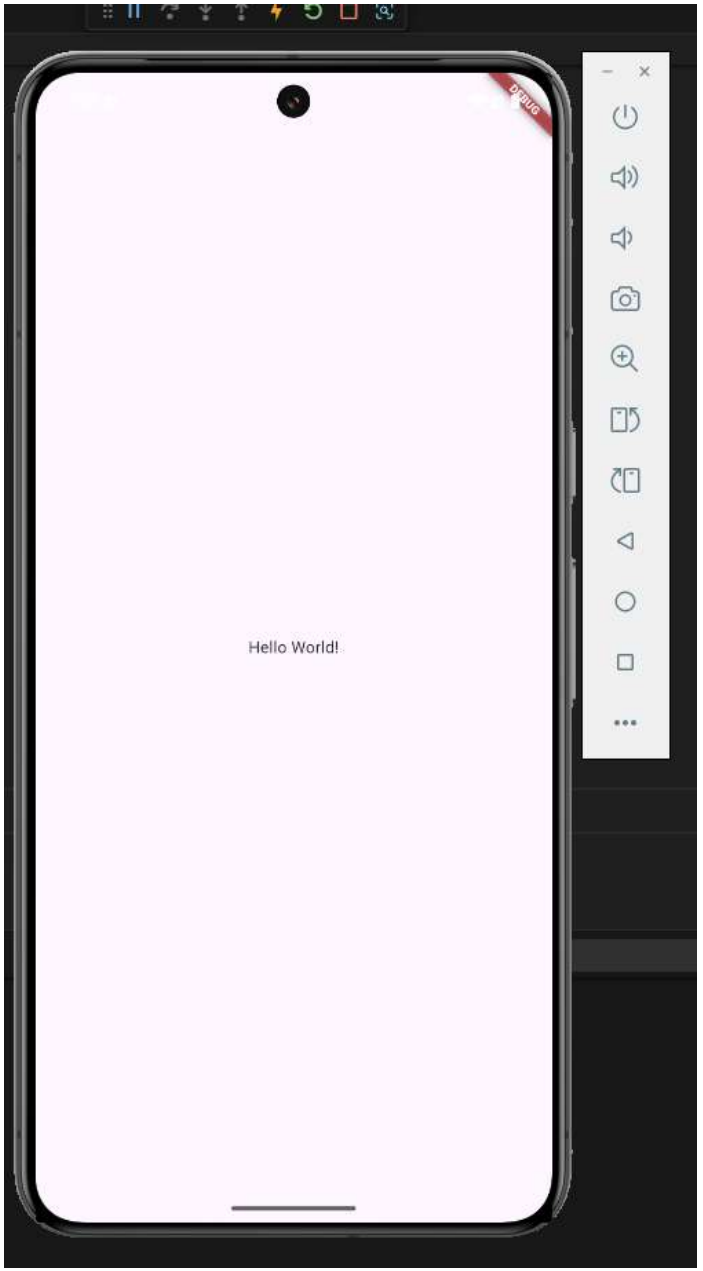
child: Text('Hello World!'): Exibe o texto "Hello World!" no centro da tela.

CTRL+Shif+P → Flutter: Select Device






7:02
Sat, May 17
Play Store
Gmail
Photos
YouTube


DEBUG
Hello World!

Modificando o projeto

- Vamos acrescentar 2 campos e um botão
  - Coletar o nome e idade do usuário
  - Informar se é maior de idade ou não

- Modificando o main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Verificador de Maioridade',
      home: const VerificaMaioridadeScreen(),
    );
  }
}

class VerificaMaioridadeScreen extends StatefulWidget {
  const VerificaMaioridadeScreen({super.key});

  @override
  _VerificaMaioridadeScreenState createState() => _VerificaMaioridadeScreenState();
}
```

```dart
class _VerificaMaioridadeScreenState extends State<VerificaMaioridadeScreen> {
  final TextEditingController nomeController = TextEditingController();
  final TextEditingController idadeController = TextEditingController();
  String resultado = '';

  void verificarMaioridade() {
    final String nome = nomeController.text.trim();
    final int? idade = int.tryParse(idadeController.text.trim());

    if (nome.isEmpty || idade == null) {
      setState(() {
        resultado = 'Por favor, insira nome e idade válidos.';
      });
      return;
    }

    setState(() {
      if (idade >= 18) {
        resultado = 'Olá $nome, você é maior de idade.';
      } else {
        resultado = 'Olá $nome, você é menor de idade.';
      }
    });
  }
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Maioridade')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            TextField(
              controller: nomeController,
              decoration: const InputDecoration(labelText: 'Nome'),
            ),
            TextField(
              controller: idadeController,
              keyboardType: TextInputType.number,
              decoration: const InputDecoration(labelText: 'Idade'),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: verificarMaioridade,
              child: const Text('Verificar'),
            ),
            const SizedBox(height: 20),
            Text(
              resultado,
              style: const TextStyle(fontSize: 18),
            ),
          ],
        ),
      ),
    );
  }
}
```


7:52
DEBUG
Maioridade
Nome
Fabricio
Idade
34
Verificar
Olá Fabricio, você é maior de idade.

```
void irParaSegundaTela() {
 Navigator.push(
   context,
   MaterialPageRoute(builder: (context) => const SegundaTela()),
 );
}
```

Troca de tela simples

```dart
class SegundaTela extends StatelessWidget {
  const SegundaTela({super.key});


  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Segunda Tela')),
      body: const Center(
        child: Text(
          'Você está na segunda tela!',
          style: TextStyle(fontSize: 22),
        ),
      ),
    );
  }
}
```

```
  const SizedBox(height: 20),
  Text(
    resultado,
    style: const TextStyle(fontSize: 18),
  ),
   const SizedBox(height: 30),
  Center(
    child: ElevatedButton(
      onPressed: irParaSegundaTela,
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.green, // cor diferente
      ),
      child: const Text('Ir para outra tela'),
    ),
  ),
],
```

Troca de tela com parâmetros

```
void irParaSegundaTela() {
    final String nome = nomeController.text.trim();

    if (nome.isEmpty) {
      setState(() {
        resultado = 'Digite seu nome antes de ir para a outra tela.';
      });
      return;
    }

    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => SegundaTela(nome: nome),
      ),
    );
  }
```

Troca de tela com parâmetros

```dart
class SegundaTela extends StatelessWidget {
  final String nome;

  const SegundaTela({super.key, required this.nome});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Segunda Tela')),
      body: Center(
        child: Text(
          'Olá, $nome! Bem-vindo à segunda tela!',
          style: const TextStyle(fontSize: 22),
          textAlign: TextAlign.center,
        ),
      ),
    );
  }
}
```

Exercício

Tela 1 – Cadastro de Usuário

- Um formulário com os seguintes campos:
  - Nome (obrigatório)
  - Idade (obrigatório, apenas números)
  - E-mail (opcional)
- Um botão "Avançar" (com cor diferente dos demais componentes)
  - Só permite avançar se os campos obrigatórios estiverem preenchidos corretamente.
  - Validar se a idade é um número > 0.
- Ao clicar, o app deve navegar para a segunda tela e **enviar os dados inseridos**.

Tela 2 – Perfil do Usuário

• Exibe os dados enviados com uma mensagem personalizada:

- "**Olá, [nome]! Você tem [idade] anos**."
- Se o e-mail foi preenchido, mostre abaixo: "**Seu e-mail é: [email]**“

• Um botão "Editar Dados" que **volta para a primeira tela**, mantendo os dados preenchidos.
• Um botão "Limpar e Reiniciar" que volta à tela inicial e **zera os dados** (estado limpo).